Отчет о воспроизведении: Super-Convergence

Статья: L. N. Smith, N. Topin. *Super-Convergence: Very Fast Training of Neural Networks Using Large Learning Rates*. arXiv:1708.07120 (v3, 2018). Код авторов (Caffe): https://github.com/lnsmith54/super-convergence

1. Коротко о проделанно работе

- **Главный результат статьи повторился.** ResNet-56 на CIFAR-10 с одним циклом LR 0.1 → 3 → 0.1 за 10k итераций дает 92.5% (в статье 92.4%), обычный ступенчатый LR за 80k итераций дает 90.0% (в статье 91.2%). Это 8 минут на одной RTX 5070 Ti, у авторов 6 часов на 8 GPU.
- **"В 8 раз быстрее" верно только против бейзлайна авторов,** а он очень длинный, 1600 эпох. Против обычного рецепта обучения ResNet выигрыш около 2.4 раза по времени, и то за счет большого батча.
- **Выигрыш на малых данных подтвердился только на 10k картинок.** Слабый бейзлайн статьи на малых данных объясняется настройкой BatchNorm у авторов.
- **На DenseNet-40 и MNIST результат повторился,** на wide ResNet-32 только частично.
- **Оценка оптимального LR 2-6 (Fig. 3a) получается только из статистик BatchNorm.** По самим весам сети оценка 0.1-0.8.
- **Треугольник (метод LR) из статьи работает так же, как современные методы** (`OneCycleLR` в PyTorch, разогрев + косинус).

2. Что проверялось с помощью экспериментов

Утверждения из статьи:

1. LR range test: у ResNet-56 точность остается высокой вплоть до LR = 3 (Fig. 2b), и это признак того, что super-convergence возможна.
2. Один цикл 0.1 → 3 → 0.1 за 10 000 итераций дает 92.4%, это лучше ступенчатого LR за 80 000 итераций (91.2%, Fig. 1a).
3. Чем меньше обучающих данных, тем больше выигрыш: до +9.2% на 10 000 картинок (Table 1).
4. Нужен именно большой `max_lr` (Table 6), а сильный weight decay мешает (Fig. 12b), потому что большой LR и так регуляризует.
5. Оценка оптимального LR по весам на трех итерациях подряд дает 2-6 (Fig. 3a).
6. Эксперименты из приложения: длина цикла, батч, momentum, weight decay, ResNet-20 (Fig. 1b, 4b, 12b, Table 6, 7, 8).
7. Эффект есть и на других сетях и данных: wide ResNet-32, DenseNet-40, LeNet на MNIST (Table 2).

Кроме экспериментов статьи, проверено, насколько честно ее сравнение и актуален ли метод сейчас: ступенчатый LR с тем же бюджетом, обычная аугментация random crop, стандартный рецепт обучения ResNet из He et al. (2016) и методы LR, которые используют сегодня. В разделах 4.6 и 4.7 они отмечены "нет в статье". Постановка в тексте статьи описана слабо, поэтому батч, аугментации, архитектура и методы LR взяты из конфигов репозитория авторов, а для сравнения использованы 30 логов их повторных запусков.

В статье	Что там	Запуски (`results/`)	Раздел	Итог
Fig. 2b, 8a	LR range test ResNet-56	`range_it5000`, `range_it20000`	4.1	повторилось, кроме падения test loss (не видно за шумом)

В статье	Что там	Запуски (`results/`)	Раздел	Итог
Fig. 1a	1cycle 10k итераций против ступенчатого LR 80k	`clr_n50000` (3 seed), `step_n50000`	4.2	повторилось
Table 1	мало данных: 10k-40k картинок	`clr_n*`, `step_n*` (по 2 seed), `step_n*_maf999`	4.3	для 1cycle повторилось, рост выигрыша только на 10k
Fig. 1b, Table 6, 7, 8, Fig. 12b	длина цикла, `max_lr`, батч, momentum, weight decay	`clr_it*`, `clr_lr*`, `clr_bs*`, `clr_mom*`, `clr_wd*`	4.4, 4.8	повторилось, кроме явного оптимума `max_lr` около 3
Fig. 4b	ResNet-20 и ResNet-110	`clr_resnet20`, `clr_resnet20_it10000`, `step_resnet20`	4.8	ResNet-20 повторился, ResNet-110 не запускался
Fig. 3a	оценка оптимального LR	`lr_estimate.py` → `lr_estimate.json`	4.5	2-6 только со статистиками BN
Раздел 3	1cycle по тексту, с хвостом и циклическим momentum	`1cycle_seed*`, `1cycle_cm_seed*`	4.7	та же точность, что у треугольника
Table 2	wide ResNet-32, DenseNet-40, LeNet на MNIST	`wide_*`, `densenet_*`, `lenet_*`	4.9	MNIST и DenseNet повторились, wide ResNet частично
нет в статье	тот же бюджет, random crop, рецепт He et al., `OneCycleLR`, косинус	`step_n50000_it10000`, `*_crop*`, `he_crop_n50000`, `onecycle_*`, `cosine_*`	4.4, 4.6, 4.7	дополнительные проверки

Не воспроизводились из-за нехватки времени и/илм ресурсов: ImageNet (около 150 ГБ данных и около 2 суток только на ResNet-50), CIFAR-100, ResNet-110, адаптивные оптимизаторы и dropout (Fig. 9, 11), range test для ResNeXt (Fig. 13), часть строк DenseNet-40 из Table 2. Для CIFAR-100, ResNet-110 и оптимизаторов есть логи авторов, и они близки к числам статьи (например, CIFAR-100: 1cycle 68.20% и ступенчатый LR 59.27% в логе, 68.6% и 59.8% в статье). Логов 1cycle с адаптивными оптимизаторами в репозитории нет, хотя в статье написано, что их запускали.

3. Отличия нашего сетапа от авторов

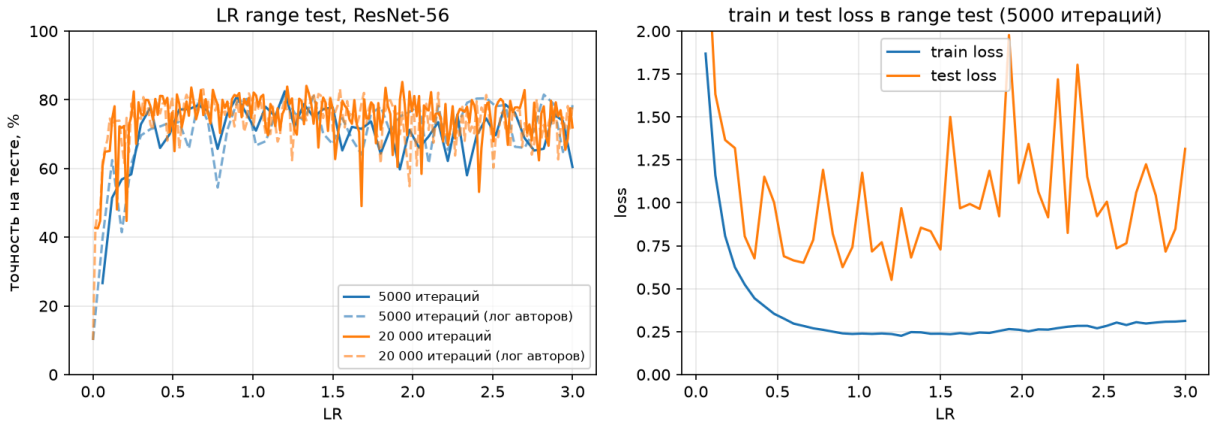
	Авторы (Caffe)	Воспроизведение (PyTorch)
Железо	8 GTX Titan Black, по 125 картинок на GPU	одна RTX 5070 Ti, батч 1000 целиком
Статистики BatchNorm	по 125 картинок на каждой GPU	по всему батчу
Накопление статистик BN (`moving_average_fraction`)	0.95 для 1cycle, 0.999 для ступенчатого LR	0.95 везде, кроме проверочных запусков (4.3)
Точность вычислений	fp32	bf16
Время одного запуска	1cycle 6 ч, ступенчатый LR 49 ч	7-8 мин и около часа
Momentum в SGD	$v = mv + \epsilon g, \theta = \theta - v$	$v = mv + g, \theta = \theta - \epsilon v$
Свертки	с bias	без bias, после них все равно BN
Тест	25 000 картинок, то есть тест 2.5 раза	ровно 10 000 картинок

Формулы momentum отличаются только при резком изменении LR, а у 1cycle LR меняется плавно. Статистики BN по 1000 картинок вместо 125, судя по совпадению с логами авторов, на итог почти не повлияли. Важная разница только одна: у авторов ступенчатый LR учился с накоплением статистик BN 0.999, и на малых данных это ломает сеть на тесте (4.3).

Архитектура ResNet-56 у авторов немного отличается от стандартной из He et al.: в блоке, где уменьшается карта, свертки не меняют число каналов, shortcut идет через avg-pool 3×3, а недостающие каналы дописываются нулями после сложения. `model.py` повторяет архитектуру авторов, 806 746 параметров (у авторов 808 682, в стандартной 853 018). На результат это не влияет: стандартный рецепт He et al. на этой архитектуре дает такую же точность, что в их статье (4.6).

4. Результаты

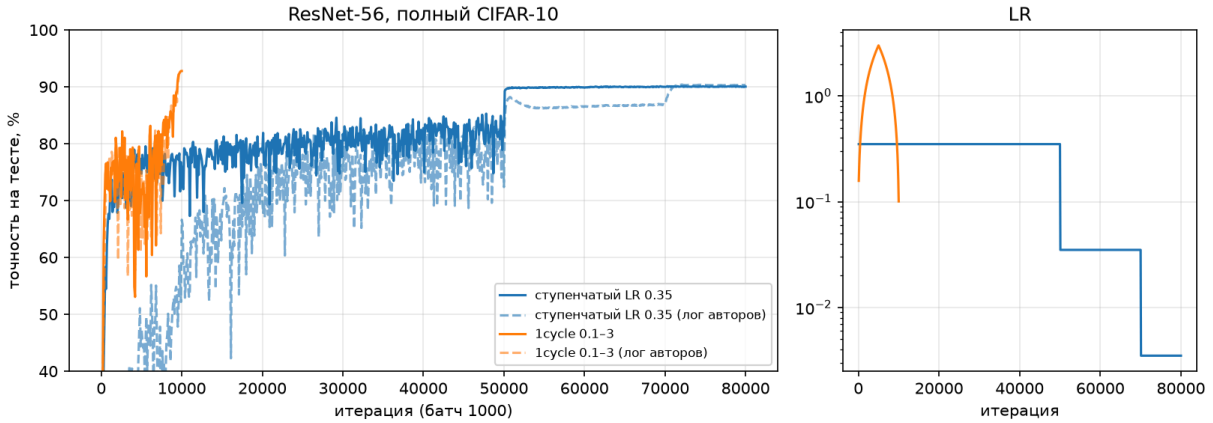
4.1. LR range test (Fig. 2b)



LR растет линейно от 0 до 3 за 5000 итераций. Точность на тесте выходит на 70-80% уже при LR около 0.3 и дальше не падает до самого LR = 3, только сильно скачет. Кривая почти совпадает с кривой авторов (максимум 82.5% против 81.6% у авторов), и на 20 000 итераций картина та же, то есть длина прогона не важна, как и пишут авторы. Единственное, что не видно: в статье (Fig. 8a) test loss при росте LR от 0.2 до 2 падает, а здесь шум между соседними точками больше этого эффекта.

Итог: повторилось, провала точности при большом LR нет.

4.2. Главный эксперимент (Fig. 1a)



Расписание LR	Итераций	Эпох	Время	Воспроизведение	Лог авторов	Статья
1cycle 0.1 → 3 → 0.1	10 000	200	7.9 мин	**92.78%**	92.44%	92.4%
ступенчатый 0.35 / 0.035 / 0.0035	80 000	1600	57.4 мин	90.03%	90.32%	91.2%

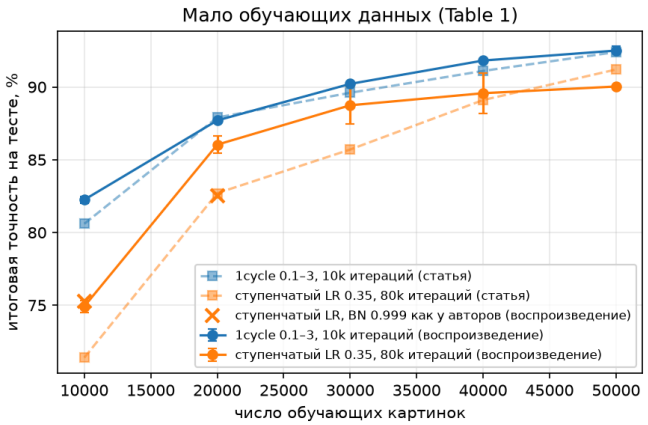
За 8 раз меньше итераций 1cycle дает на 2.7 п.п. больше, чем ступенчатый LR, и обе точности отличаются от логов авторов меньше чем на 0.4 п.п. По трем запускам с разным seed 1cycle дает 92.50 ± 0.24%. Кривая 1cycle необычная: пока LR большой, точность на тесте скачет от 53 до 85%, а весь рост до 92-93% происходит на спуске LR, в последние ~2000 итераций. У ступенчатого LR после первого снижения точность сразу 89.8% и дальше почти не растет, то есть последние 600 эпох почти ничего не дают.

Итог: главный результат статьи повторился.

4.3. Мало обучающих данных (Table 1)

Картинок	1cycle: seed 0 / 1	1cycle: лог авторов / статья	ступенчатый: seed 0 / 1	ступенчатый, BN 0.999	ступенчатый: лог авторов / статья	выигрыш 1cycle	выигрыш в статье
10 000	82.47 / 82.03	80.59 / 80.6	74.52 / 75.32	75.27	71.20 / 71.4	**+7.33**	+9.2
20 000	87.81 / 87.59	87.92 / 87.9	86.63 / 85.44	82.52	62.33 / 82.7	+1.67	+5.2
30 000	90.17 / 90.25	89.64 / 89.6	90.04 / 87.44	нет	86.36 / 85.7	+1.47	+3.9
40 000	91.82 / 91.81	91.10 / 91.1	88.18 / 90.96	нет	90.43 / 89.1	+2.25	+2.0
50 000	92.50 (3 seed)	92.44 / 92.4	90.03	нет	90.32 / 91.2	+2.47	+1.2

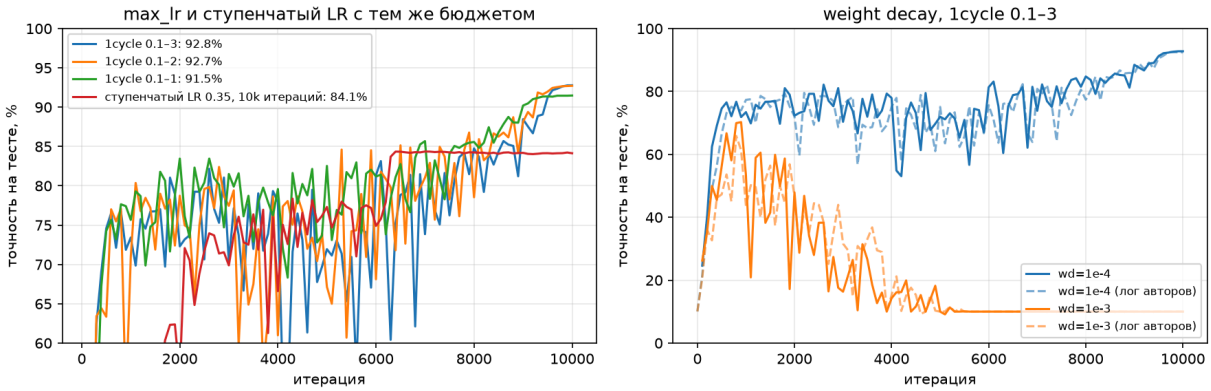
Обучение идет на одной и той же случайной части CIFAR-10 тем же числом итераций, тест на всех 10 000 картинках. Для 1cycle все точки совпали со статьей (от -0.2 до +1.7 п.п.), а ступенчатый LR сильно скачет от запуска к запуску, до 2.8 п.п. между seed. 1cycle лучше на всех размерах, но выигрыш большой только на 10k картинок (+7.3 п.п.). На 20k-50k он около +1.5-2.5 п.п., а в статье плавно растет до +9.2.



Почему у авторов бейзлайн слабее, видно по их логам. Их ступенчатый LR учился с накоплением статистик BN 0.999 (это есть даже в самой Table 1). На 10k картинок их точность на тесте до 48k-й итерации держится около 10%, хотя на обучающей выборке почти 100%: сеть не расходится, она ломается только на тесте, потому что накопленные статистики BN не успевают за весами. Бейзлайн на 10k и 20k картинок был повторен с BN ровно как в Caffe (флаг `--bn-maf 0.999`), и провал повторился: на 10k картинок с 20k по 52k итерацию точность на тесте почти все время около 10%, после снижений LR она поднимается до 62.5% и 75.3% (у авторов 55-60% и 71%). На 20k картинок итог 82.52%, почти ровно как в статье (82.7%), против 85.4-86.6% с обычной BN. Оговорка одна: статистики BN не влияют на сами веса, поэтому по одному запуску нельзя отделить потерю от BN от обычного разброса между запусками.

Итог: для 1cycle повторилось. Провал бейзлайна у авторов это настройка BN, а не сам ступенчатый LR, поэтому с обычной BN выигрыш 1cycle на 20k-40k картинок меньше, чем в статье.

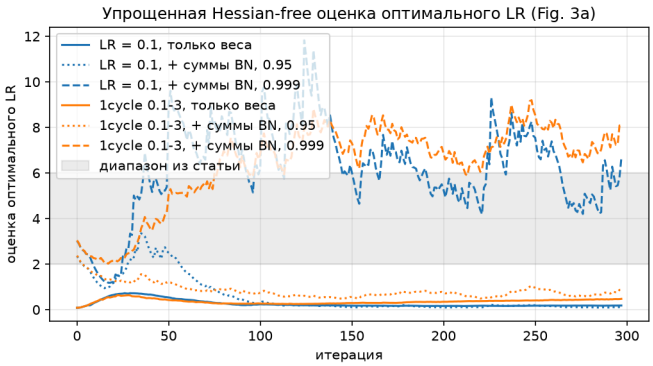
4.4. Варианты главного запуска



Запуск (весь CIFAR-10, 10k итераций)	Воспроизведение	Статья / лог авторов
1cycle 0.1-3, три seed	92.78 / 92.38 / 92.34, среднее **92.50 ± 0.24**	92.4
1cycle 0.1-2 и 0.1-1	92.72 и 91.48	91.7 и 91.3 (Table 6)
1cycle 0.1-3, weight decay 10^{-3}	**10.00** (лучшая за запуск 70.3)	9.95 в логе авторов (Fig. 12b)
ступенчатый LR 0.35, тот же бюджет, два seed	84.14 / 84.46	нет

С `max_lr` = 1 точность примерно на 1 п.п. ниже, а между 2 и 3 разницы нет. Weight decay 10^{-3} полностью ломает обучение: точность растет до 70%, потом падает, и к 5000-й итерации сеть всегда выдает один класс. Кривая почти такая же, как у авторов, и это подтверждает их мысль, что при LR = 3 сильный weight decay уже лишний. Ступенчатый LR с тем же бюджетом в 10k итераций дает только 84.1-84.5% против 92.5% у 1cycle, значит, дело не только в том, что бейзлайн авторов очень длинный.

4.5. Оценка оптимального LR (Fig. 3a)



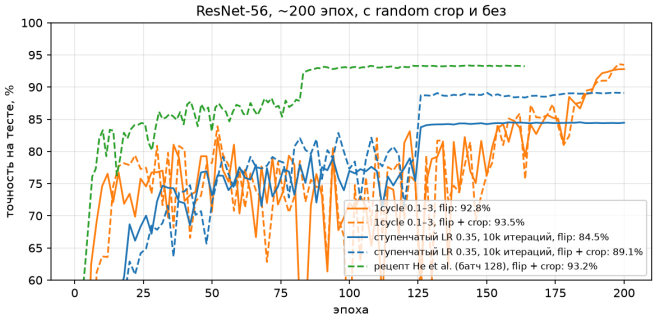
Если считать по формуле статьи только по весам сети, оценка получается 0.1-0.8, а не 2-6. В скрипте авторов `L2TermComputation3.py` формула та же, но сумма идет по всем тензорам Caffe, в том числе по слоям BatchNorm. А у BatchNorm в Caffe лежат не веса, а накопленные суммы статистик и счетчик, которые растут на каждой итерации. Если посчитать эти суммы ровно как в Caffe и добавить в формулу, то при накоплении 0.95 оценка в начале поднимается до 2-3, но к 300-й итерации падает до 0.1-0.9. При накоплении 0.999 суммы растут все 300 итераций, и оценка держится на уровне нескольких единиц: к 300-й итерации 3.4, 5.2 и 6.7 в трех запусках, а в одном запуске весь диапазон ровно 2.1-5.6. Это и есть диапазон 2-6 из статьи. Число при этом очень шумное и от запуска к запуску сильно меняется.

Итог: число 2-6 получается, только если, как в скрипте авторов, сложить веса вместе с накопленными статистиками BatchNorm. По самим весам оценка 0.1-0.8. К тому же формула не учитывает

momentum, а все запуски идут с momentum 0.9. Поэтому как довод за большой LR эта оценка не работает.

4.6. Random crop и стандартный рецепт (нет в статье)

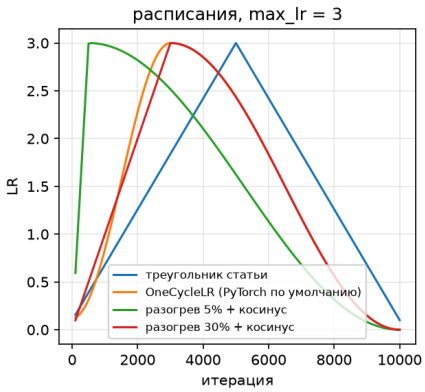
Запуск	Батч	Эпох	Аугментация	Время	Точность
1cycle 0.1-3, 10k итераций	1000	200	flip	7-8 мин	92.50 (3 seed)
ступенчатый LR 0.35, 10k итераций	1000	200	flip	7.3 мин	84.14 / 84.46
1cycle 0.1-3, 10k итераций	1000	200	flip + crop	7.3 мин	**93.45**
ступенчатый LR 0.35, 10k итераций	1000	200	flip + crop	7.3 мин	89.09
рецепт He et al., 64k итераций	128	164	flip + crop	17.8 мин	**93.23** (в их статье 93.03)

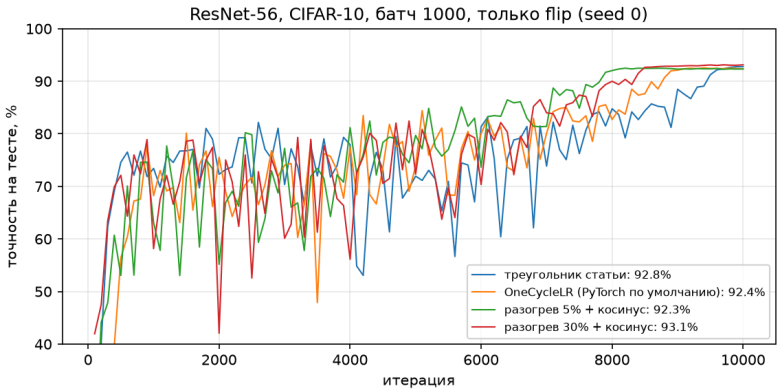


В статье из аугментаций только flip, и сеть доводит точность на обучающей выборке до 100%. Здесь проверено, остается ли выигрыш 1cycle с обычной аугментацией random crop, и он сравнен со стандартным рецептом обучения ResNet-56 из He et al. (2016). С crop обе схемы лучше, но ступенчатому LR crop помогает сильнее (+4.8 п.п. против +1 п.п.), и разрыв при равном бюджете сокращается с 8 до 4.4 п.п. Это сходится с мыслью статьи: когда регуляризации от аугментации больше, большой LR дает меньше. Рецепт He et al. в этом коде дает 93.23%, как в их статье, а 1cycle с crop примерно столько же, 93.45%. По эпохам 1cycle даже дольше (200 против 164), но по времени в 2.4 раза быстрее, потому что большой батч 1000 GPU считает гораздо эффективнее батча 128, а большой LR позволяет учить с таким батчем без потери точности.

Итог: против стандартного рецепта 1cycle выигрывает по времени в 2.4 раза, а не в 8.

4.7. Треугольник статьи против современных расписаний (нет в статье)





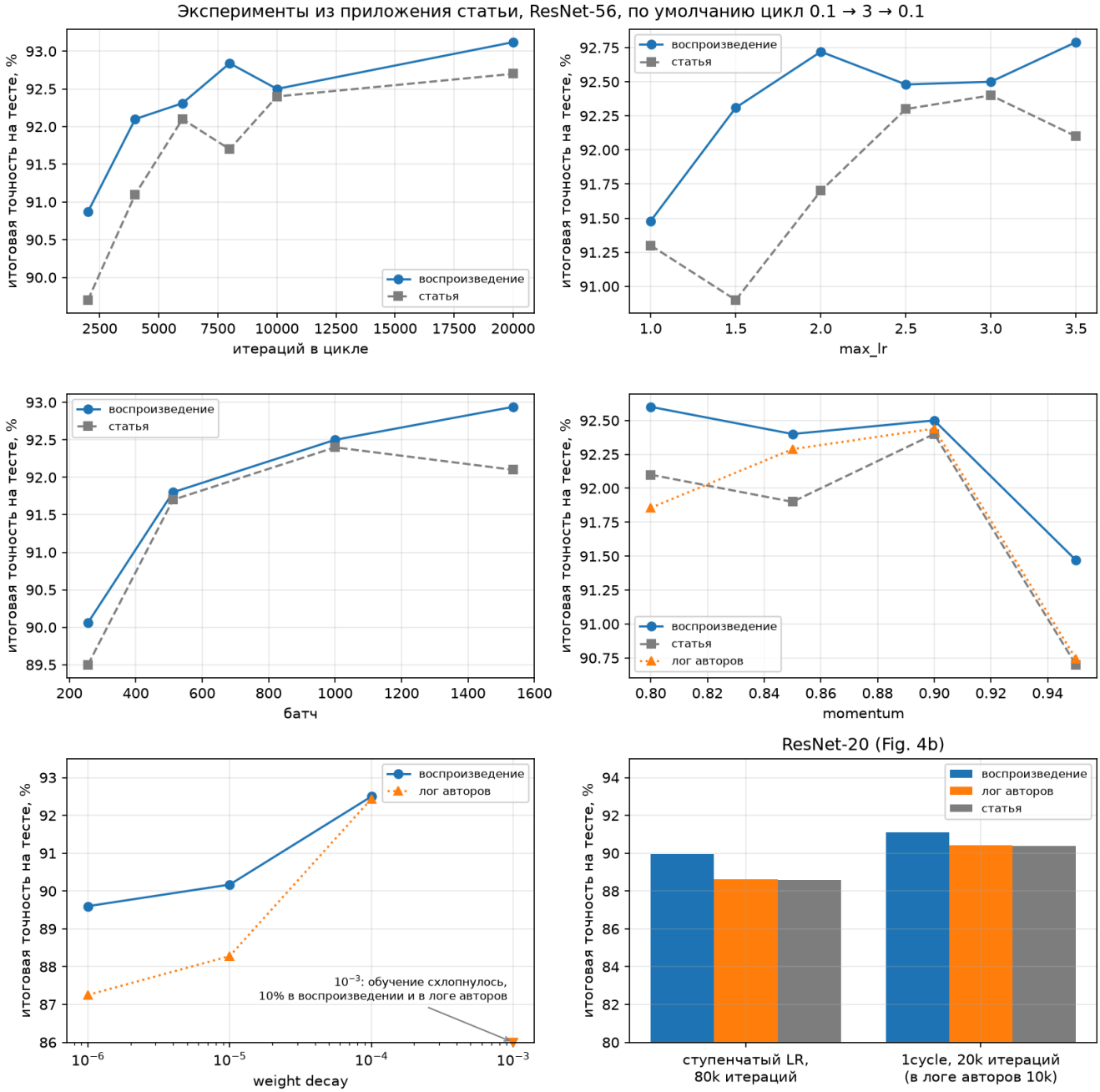
Расписание	LR	Momentum	flip	flip + crop
треугольник статьи	0.1 → 3 → 0.1	0.9	среднее **92.50** (3 seed)	93.45
`OneCycleLR` из PyTorch	по косинусу 0.12 → 3 за 30%, потом почти до нуля	0.95 → 0.85 → 0.95	среднее **92.22** (2 seed)	93.56

Расписание	LR	Momentum	flip	flip + crop
разогрев 5% + косинус	$0 \to 3$ за 500 итераций, потом по косинусу до 0	0.9	среднее **92.46** (2 seed)	**93.90**
разогрев 30% + косинус	$0 \to 3$ за 3000 итераций, потом по косинусу до 0	0.9	93.12 (1 seed)	нет
1cycle по тексту статьи	треугольник на 90% итераций, потом до 10^{-4}	0.9	среднее **92.68** (2 seed)	нет
то же с циклическим momentum	то же	$0.95 \to 0.85 \to 0.95$	среднее **92.35** (2 seed)	нет

Здесь все как в главном эксперименте, меняется только расписание LR. С одним flip все расписания дают одно и то же: разница меньше разброса между seed. 1cycle точно по тексту статьи, с хвостом и циклическим momentum, тоже ничего не добавил, так что авторы ничего не теряли, когда брали простой треугольник. Косинус с разогревом на 30% и с random crop выглядят чуть лучше, но это по одному запуску. По кривым видна разница: у косинуса точность выходит на 92% уже к 7000-8000-й итерации, а у треугольника рост только в последние ~2000 итераций.

Итог: важны большой LR в середине и плавный спуск в конце, а не точная форма расписания.

4.8. Эксперименты из приложения статьи



длина цикла, итераций	2k	4k	6k	8k	10k	20k
статья	89.7	91.1	92.1	91.7	92.4	92.7
воспроизведение	90.87	92.10	92.31	92.84	92.50	93.12
`max_lr`	**1.0**	**1.5**	**2.0**	**2.5**	**3.0**	**3.5**
статья	91.3	90.9	91.7	92.3	92.4	92.1
воспроизведение	91.48	92.31	92.72	92.48	92.50	92.79

батч (эпох за 10k итераций)	256 (51)	512 (102)	1000 (200)	1536 (307)
статья	89.5	91.7	92.4	92.1
воспроизведение	90.06	91.80	92.50	92.94
momentum	**0.80**	**0.85**	**0.90**	**0.95**

батч (эпох за 10k итераций)	256 (51)	512 (102)	1000 (200)	1536 (307)
статья / лог авторов	92.1 / 91.86	91.9 / 92.29	92.4 / 92.44	90.7 / 90.74
воспроизведение	92.60	92.40	92.50	91.47
weight decay	10^{-3}	10^{-4}	10^{-5}	10^{-6}
лог авторов	9.95	92.44	88.28	87.25
воспроизведение	10.00	92.50	90.17	89.60

Везде ResNet-56, по одному запуску на точку. Почти все повторяет статью: чем длиннее цикл, тем выше точность (даже цикл на 2k итераций, всего 40 эпох, дает 90.87%, больше, чем ступенчатый LR за 80k итераций); momentum от 0.8 до 0.9 почти без разницы, а 0.95 хуже; weight decay 10^{-3} ломает обучение, а 10^{-5} и 10^{-6} стоят 2-3 п.п. точности. Не повторился только явный оптимум `max_lr` около 3: от 1.5 до 3.5 все значения в пределах 0.5 п.п. У эксперимента с батчем есть подвох, о котором статья не говорит: число итераций фиксировано, поэтому с батчем 256 сеть видит 51 эпоху, а с батчем 1000 уже 200, и часть выигрыша большого батча это просто больше эпох.

ResNet-20 (Fig. 4b) тоже повторился: 1cycle дает 91.12%, ступенчатый LR 89.94% (у авторов 90.4% и 88.6%). В логе авторов 1cycle шел 10k итераций, а не 20k, как написано в статье, поэтому проверены оба варианта, и они дают одно и то же (91.08% и 91.12%).

Итог: повторилось, кроме оптимума `max_lr` около 3.

4.9. Другие сети и MNIST (Table 2)

Сеть	Расписание	Эпох	Время	Воспроизведение	Статья
wide ResNet-32	1cycle 0.1 → 0.5 / 0.1 → 1 / 0.1 → 1	25 / 50 / 100	1.3 / 2.4 / 4.9 мин	83.64 / 88.93 / **90.03**	87.3 / 91.3 / **91.9**
	ступенчатый LR 0.1	100 / 200 / 400 / 800	4.9 / 9.6 / 19.3 / 37.1 мин	83.68 / 87.66 / 87.68 / **90.58**	86.7 / 88.7 / 89.8 / 90.3
DenseNet-40	1cycle 0.1 → 4, wd 10^{-6}	100	21.3 мин	**92.62**	92.2
	ступенчатый LR 0.1, wd 10^{-4}	200	42.5 мин	92.16	92.1
LeNet, MNIST (3 seed)	стандартный `inv` / ступенчатый LR	85	19 с	99.07 / 99.09	99.03 / 99.00
	1cycle	12 / 25 / 50 / 85	3-19 с	99.11 / 99.16 / 99.16 / **99.24**	99.25 / 99.28 / 99.27 / **99.35**

Настройки взяты из Table 2: батч 512, диапазоны LR и momentum, weight decay и число эпох; после цикла LR падает до 1/1000 нижней границы, как в разделе 3 статьи. Чего в статье нет, пришлось выбрать самостоятельно. Когда снижать LR у ступенчатых бейзлайнов, статья не пишет, поэтому LR снижался в 10 раз на 50% и 75% обучения, как в рецептах DenseNet и wide ResNet. wide ResNet-32 описан одной фразой ("число каналов растет в 4 раза вместо 2"), поэтому взяты блоки ResNet авторов с ростом каналов 16 → 64 → 256. DenseNet-40 сделан по коду DenseNetCaffe, на который ссылается статья, а LeNet и его стандартное расписание `inv` взяты из примера MNIST в Caffe. Для wide ResNet и DenseNet по одному запуску на строку, для MNIST по три.

DenseNet-40 повторился: 1cycle за 100 эпох дает ту же точность, что ступенчатый LR за 200 эпох, в 2 раза быстрее. На MNIST картина та же, но эффект меньше: 1cycle за 12 эпох на уровне стандартного расписания за 85 эпох, но не лучше, как в статье, и все 1cycle здесь примерно на 0.1 п.п. ниже статьи. wide ResNet-32 повторился частично: 1cycle за 100 эпох лучше ступенчатого LR за 100-400 эпох, но

чуть хуже ступенчатого за 800 эпох, а в статье лучше всех. 1cycle здесь на 2-4 п.п. ниже статьи. Причин может быть несколько: архитектура описана одной фразой, неизвестно, когда авторы снижали LR, и здесь по одному запуску, а в статье разброс для 800 эпох ±1.0 п.п.

Итог: DenseNet-40 и MNIST повторились, wide ResNet-32 частично.

5. Выводы

1. **Главный результат статьи повторился почти точно:** 92.5 ± 0.2% у 1cycle за 10k итераций против 92.4% в статье, ступенчатый LR авторов за 80k итераций 90.0% против 91.2% в статье и 90.3% в их логе. Где результат повторился, числа совпадают со статьей и логами авторов в пределах 0.1-2.4 п.п., а кривые похожи на кривые из логов. Это во многом заслуга авторов: без их конфигов и логов нельзя было бы узнать ни про батч 1000, ни про то, что нет random crop.
2. **LR range test, нужность большого `max_lr` и вред сильного weight decay тоже повторились,** как и эксперименты из приложения и ResNet-20. Не подтвердился только явный оптимум `max_lr` около 3.
3. **"В 8 раз быстрее" верно только против бейзлайна авторов в 1600 эпох,** из которых около 600 почти ничего не дают. Против стандартного рецепта обучения ResNet выигрыш 2.4 раза по времени, за счет большого батча. Но при равном бюджете 1cycle намного лучше ступенчатого LR (92.5% против 84.3%), так что форма расписания действительно важна.
4. **Выигрыш на малых данных подтвердился только на 10k картинок.** Провал бейзлайна в логах авторов повторяется с их настройкой BatchNorm, то есть самый большой выигрыш статьи отчасти получен за счет неудачной настройки BN у бейзлайна.
5. **На DenseNet-40 и MNIST 1cycle дает ту же точность за в 2-7 раз меньше эпох,** на wide ResNet-32 результат повторился частично.
6. **Оценка оптимального LR 2-6 получается из накопленных статистик BatchNorm, а не из весов.** По весам оценка 0.1-0.8, так что как довод за большой LR она не работает.
7. **Треугольник из статьи не хуже современных расписаний,** и 1cycle точно по тексту статьи тоже. Важны большой LR и плавный спуск, а не форма. Встроенный разогрев, видимо, и позволяет 1cycle спокойно работать с LR = 3.
8. **На практике статья полезна:** 1cycle с большим LR дает хорошую точность за 8 минут на одной обычной видеокарте, а `max_lr` удобно подбирать LR range test.

6. Источники

Источник	Что оттуда взято
Статья, arXiv:1708.07120 v3	заявленные числа, формулы CLR, 1cycle и оценки LR, железо, батч 1000 (сноска 2)
Репозиторий авторов lnsmith54/super-convergence	расписания LR, архитектура, батч, аугментации, настройка BN, скрипт оценки LR `L2TermComputation3.py`, 30 из 43 логов их запусков
K. He et al. Deep Residual Learning for Image Recognition (2016)	стандартный рецепт обучения ResNet-56 и его точность 93.03%
liuzhuang13/DenseNetCaffe и пример MNIST из Caffe	архитектуры DenseNet-40 и LeNet, расписание `inv`
Документация PyTorch и исходники Caffe (`sgd_solver.cpp`, `batch_norm_layer.cpp`)	формулы momentum, как копятся статистики BatchNorm, настройки `OneCycleLR`